Gebze Technical University
Department of Computer Engineering
CSE 241/501
Fall 2024
Final Exam Jan 8 2025

Student Code	Q1 (18)	Q2 (25)	Q3 (45)	Q4 (12)	Exam Format (-10)
1180					

Read the instructions below carefully

- Do not write your name or your student number in any part of the exam materials including the answer sheets.
- Your individual student codes are already printed on your question and answer sheets. Keep this student code until the exam ends. You will use it to sign the exam attendance sheet.
- Answer each question on the designated page in the box only. We will not (and cannot) grade any materials outside of the designated boxes.
- Hand over your cell phones, books, and notes to the TA before the exam starts. You will not use any electronic devices or books during the exam.
- This exam has 4 questions; it will be graded out of 100 points. You have 180 minutes. If you do not follow any of the format rules, you will lose 10 points.
- After the exam, you will receive your scanned exam papers by email. You will type your answers on an online form. Your exam will not be graded if you do not provide typed answers. This task is a part of the final exam so your typed answers should exactly match your scanned exam paper.

Question 1

a) Discuss the differences between manual memory management in C++ and automatic garbage collection in Java.

b) How does the use of move semantics improve the performance of the standard library containers such as `std::vector` or `std::list`?

c) What are the similarities and differences between `equals` method of `Object` class and == operator in Java?

Question 2

a) Write a `Product` class in Java to represent items with a unique `ID` (an integer), a `name` (a string), and a `price` (a double). Your class should include the following methods and features:

- A constructor that initializes the attributes (throw an exception if the `ID` is negative or the `price` is invalid).
- Getters and setters for all attributes (validate the `ID` and `price` to ensure they remain valid).
- A method `applyDiscount(double percentage)` that reduces the price of the product by the given percentage (ensure the resulting price is non-negative).
- A static method `findCheapest(Product[] products)` that takes an array of `Product` objects and returns the one with the lowest price.
- An overridden `equals` method that considers two products equal if their `ID` is the same (ignore `hashcode`).
- An overridden `toString` method to generate a string representation of the product.

b) Write another class with a `main` method to demonstrate the functionality of the `Product` class:

- Create an array of at least five `Product` objects with varying IDs, names, and prices.
- Use an enhanced for loop to apply a discount to each product.
- Find and print the details of the cheapest product using the `findCheapest` method.
- Demonstrate the use of the `equals` method by comparing two products.

Question 3

You are tasked with designing a class, `StudentList`, that stores information about students in a simple array. The class should provide functionality to iterate through the students and manage the underlying data efficiently. Below is the partial implementation:

```
#include <string>

class StudentList {
private:
    struct Student {
        std::string name;
        int grade;
    };
    Student* students;
    std::size_t sz;
    std::size_t capacity;

public:
    StudentList(std::size_t initialCapa-
city);
    ~StudentList();

    void addStudent(const std::string&
name, int grade);
    std::size_t size() const;

    // Nested Iterator class
    class Iterator {
        // To be implemented
    };

    Iterator begin() const;
    Iterator end() const;
};
```

a) Implement the `Iterator` class for `StudentList`.

The iterator should include the following:

- A constructor to initialize the iterator with the `StudentList`'s data and a starting position.
- Implementations of `operator*` (to access the current `Student`), `operator++` (both pre- and post-increment), `==`, and `!=`.
- A method to retrieve the name and grade of the current student (`operator*` should return a `Student&` object).

---

b) Implement the following for the `StudentList` class:

1. **Constructor and Destructor:** Properly allocate and deallocate memory for the array of students.
2. **Copy and Move Semantics:** Implement the copy constructor, move constructor, copy assignment operator, and move assignment operator to handle resource management for the dynamically allocated student array.
3. **`addStudent`:** Add a new student to the list. If the capacity is exceeded, allocate a new array with double the capacity, move the old data to the new array, and deallocate the old memory.

---

c) Write a `main` function that:

1. Creates a `StudentList` with an initial capacity of 3.
2. Adds students `{ "Alice", 85 }`, `{ "Bob", 90 }`, `{ "Charlie", 78 }`, and `{ "Diana", 92 }`.
3. Uses only the `Iterator` class to:
    - Print all students' names and grades.
    - Find the student with the highest grade and print their details.

Question 4

What is the output of the following program on screen?

```
#include <iostream>
#include <memory>
class Base {
public:
    int value;
    Base() : value(100) {
        std::cout << "Base constructor!\n";
    }
    Base(const Base& o) : value(o.value) {
        std::cout << "Base copy constructor!\n";
    }
    Base(Base&& o) : value(std::move(o.value)) {
        std::cout << "Base move constructor!\n";
    }
    virtual ~Base() {
        std::cout << "Base destructor!\n";
    }
    virtual void display() const {
        std::cout << "Base: " << value << "\n";
    }
};
class Derived : public Base {
public:
    int extraValue;
    Derived() : Base(), extraValue(50) {
        std::cout << "Derived constructor!\n";
    }
    Derived(const Derived& o) : Base(o),
        extraValue(o.extraValue) {
        std::cout << "Derived copy constructor!\n";
    }
    Derived(Derived&& o) : Base(std::move(o)),
        extraValue(std::move(o.extraValue)) {
        std::cout << "Derived move constructor!\n";
    }
    ~Derived() {
        std::cout << "Derived destructor!\n";
    }
    void display() const override {
        std::cout << "Derived: " << value
                  << ", Extra: " << extraValue
                  << "\n";
    }
};
std::unique_ptr<Base> createObject() {
    std::cout << "Creating Derived object...\n";
    return std::make_unique<Derived>();
}
int main() {
    std::cout << "Part 1\n";
    auto b1 = std::make_unique<Derived>();
    b1->display();
    std::cout << "Part 2\n";
    auto b2 = createObject();
    b2->display();
    std::cout << "Part 3\n";
    Derived d1, d2 = std::move(d1);
    std::cout << "Part 4\n";
    b1 = std::move(b2);
    std::cout << "End of program\n";
    return 0;
}
```

Write your answer for **Question 4** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

Part 1
Base constructor!
Derived constructor!
Derived: 100, Extra: 50
Part 2
Creating Derived object...
Base Constructor!
Derived constructor!
Derived: 100, Extra: 50
Part 3
Base constructor!
Derived constructor!
Base move constructor!
Derived move constructor!
Part 4!
Derived Constructor;
Base Destructor!
Derived Destructor
Base Destructor!
Derived Destructor!
End of program

Write your answer for **Question 1** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

Q1-a

Manuel memory monogement in C++ has a lot of problem them automatic garbage collection in Java. Firstly in manuel we decide the which memory will we use. So we can reach the important memory for computer and it can be a big mistake for computer. But in Java automatic garboge collection is firstly automatic and Java Virtual Machine decides all process so it is more trustly then manuel memory monogement. Other side in manuel memory monogement in C++ we can forget the delete the pointers then it can couse memory leak or dangling reference. But in Java garboge collector automaticly delete the pointers and avoids problem. Finally automatic garboge collection more trustly then manuel memory monogement in C++.

Q1-b

Move semantics work like take values from that memory and put here. So this is so logically process.

In standart library containers such as std::vector and std::list move semantks work effecientfy. Before will take the index of vector after that reach the memory of that vector. And finally take that value.

for list it is some like vector. Take index take value and evaluate the processes.

Q1-c

In differences equals method of object class taking reference of object and looking memory of object and after that compare the objects. But == operator looking directly value of object. for exomple == operator looking just size == 0.size but equals method reach the adress of object.

Similarities both of them reach the correct value and then evaluate the process.

Write your answer for **Question 2-a** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
package mypackage;

public class Product {

private int ID;
private String name;
private double price;

public Product(int nID, String nname
, double nprice) throws Exception {
if(ID<0){
new throw negative("ID is can
not be a negative!");
ID=0 }
ID = nID;

if( price < 0.0 ) {
new throw invalid("Price is can
not be less than 0.0!");
price = 0.0;
}
price = nprice;

name = nname;

}
public int getID() {
return ID;
}
public double getPrice() {
return price;
}
public String getName() {
return name;
}
public void setID(int nID)throws
Exception {
  if(nID<0) {
new throw negative("ID is can
not be a negative!");
ID=0; }
ID = nID;
}

public void setPrice(double nPrice)
throws Exception {
if ( nPrice < 0.0) {
new throw invalid("Price is can
not be less then 0.0!");
price = 0.0 }
price = nPrice;
}
public void setName(String nname){
name = nname; }

public void applyDiscount(double
percentage)throws Exception {
price=price-(price*percentage
/100.0);
if(price < 0.0) { new throw
invalid("price can not be less
then 0.0!");
 price = 0.0; }
}
public static double findCheapest
(Product[] products)throws Exception {
double lowest = 0.0;
double x;
  for( x : products) {
   if( x < lowest) {lowest = x; }
}
  if(lowest < 0.0) { new throw
invalid("Price is can not be less
then 0.0!");
 lowest = 0.0;}
return lowest;
}
public bool equals(Product x, Product
y) {
return  x.ID == y.ID; }
public String toString(){
return String.format("Product
name is %s has %d ID and
It's price is = %.1f", name, ID
, price);
}

} //end of class Product
```

STUDENT CODE:1180

Write your answer for **Question 2-b** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
import mypackage.Product;

public static void main(String args[]
{
Product prd1=new Product(111,
"ABC", 11.1);
Product prd2= new Product(222,
"DEF", 22.2);
Product prd3=new Product(333,
"GHI", 33.3);
Product prd4=new Product(444,
"JKL", 44.4);
Product prd5= new Product(555,
"MNO", 55.5);

Product array =new Product[5];
array[0]= prd1;
array[1]=prd2;
array[2]=prd3;
array[3]=prd4;
array[4]= prd5;

System.out.println("Price of products
before discount");

for(int i=0; i<array.length;i++){
systemout.printf("Product :%s =%lf
\n", array[i].getNome(), array[i].
getPrice());
}
prd1.applyDiscount(10.0);
prd2.applyDiscount(20.0);
prd3.applyDiscount(30.0);
prd4.applyDiscount(40.0);
prd5.applyDiscount(50.0);

System.out.println("Price of products
after the discount");
for(int i=0; i<array.length;i++){
System.out.printf("Product %s=%lf
\n", array[i].getNome(), array[i].get
Price());
```

```
double cheapest = findCheapest
(Product [] array);
 int lowest_index;
for(int i=0; i<array.length;i++){
for(int n: array[i].price){
 if(n==lowest){
 lowest_index=i;}
}
}
System.out.println("Cheapest product
is %s and it's price is =%lf\n",
array[i].nome, array[i].price);

if(equals(prd1, prd2){
System.out.println("Those products
are equal!");}
else{
system.out.println("Those products
are not equal");

} //end of main
```

STUDENT CODE:1180

Write your answer for **Question 3-a** in the box below. Do not write anywhere outside the box. Follow the line guides when writing.

```
#ifndef Iterator_h
#define Iterator_h
#include "Student.h"
class Iterator :public StudentList{
privete
StudentList data;
int index;
public:

Iterator(const StudentList &o,
int nindex):index(nindex){

data.sz=0.sz;
data.capacity=0.capacity;
data.students=new Student[
capacity];

for(int i=0;i<sz;;++){
data.students[i]=0.students[i];
}

}

Student operator*(){

return (data.students[index])
}

bool operator==(const Student
&o){

if(data.students[index].nome
==o.Student.nome &&
data.students[index].grade==
Student.grade){
  return true;
}
return false;
}

bool operator!=(const Student
&o){
if(data.students[index].nome==
student.nome && data.students[index]
.grade==Student.grade){
  return false;}
  return true;
}

Student operator++(int){
return (data.students[index
++]);

Student &operator++(){
index++
return (data.students[index];

}

#endif
```

STUDENT CODE:1180

Write your answer for **Question 3-b** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
#ifndef Student_h
#define Student_h
#include <String>
using namespace std;
#include <iterator>
#include <iostream>

class StudentHist {
private:
  struct Student {
    string name;
    int grade;
  };
  Student * students
  size_t sz;
  size_t capacity;

public :

StudentHist(size_t initialCapacity)
: capacity(initialCapacity), sz(0) {
 students = new Student[capacity];
}

~StudentHist() {
 delete [] students;
}
StudentHist(const StudentHist &o) {
students = new Student[o.capacity];
capacity = o.capacity;
sz = o.sz
for(int i=0; i<sz; i++) {
students[i] = o.students; }
}

StudentHist & operator = (cons:
StudentHist &o) {
if(this!=&o) {
sz = o.sz;
capacity = o.capacity;
delete [] students;
for(int i=0; i<sz; i++) {
students[i] = o students[i]; }
}
 return *this;
}
StudentHist(StudentHist &&o) noexcept{
sz = o.sz;
capacity = o.capacity;
students = new Student[capacity];
students = move(o.students);
o.sz = 0;
o.capacity = 0;
o.students = nullptr;
}

StudentHist & operator = (StudentHist
&&o) noexcept {
if(this!=&o) {
sz = o.sz;
capacity = o.capacity;
delete [] students;
students = new Student[capacity];
for(int i=0; i<sz; i++) {
students[i] = o.students[i]; }
o.sz = 0;
o.capacity = 0;
o.students = nullptr;
}
return *this;
}

void addStudent(const string
&name, int grade) {
if(size >= capacity) {
capacity *= 2; }
StudentHist temp;
temp.sz = sz;
temp.capacity = capacity;
temp.students = move(o.students);
students = nullptr;
temp.students[sz].name = name;
temp.students[sz].grade = grade;
students = new Student[temp.capacity];
sz = temp.sz;
sz++;
}
```

Continue your answer for **Question 3-b** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
int getSize() const {

return sz;
}
```

Write your answer for **Question 3-c** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
#include "student.h"
#include "Iterator.h"

StudentList s1 =new StudentList(3);

s1.addStudent("Alice", 85);
s1.add Student ("Bob", 90);
s1.add Student("Charlie, 78);
s1.odd Student ("Diana", 92);

Iterator a=new Iterator(s1, 0);
int h-grade=0
int x;
for(int i=0; s1.getSize(); ;++){

if(h-grade <=s1*.grade) {
h-grade = s1*.grade;
x=i;
}
```

```
cout<<"Highest grade student
is << s1.students[x].name<<
has << s1.students[x].grade<<endl;
```

Continue your answer for **Question 3-c** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
cout<< "All students and
grades are"<< endl;

for(int i=0; i< s1.sz; i++){

cout<<student << s1.students[i]
name << has << s1.students[i].
grade<< endl;
  return 0;
}
```